

Workflow and Retrospective

A detailed n8n Cloud lab for routing unsupported answers, human review, monitoring, controlled failure, and rollback

EXPECTED TIME	EXPECTED EXECUTIONS	TEAM MODE
50–60 minutes	6–8 total	Each student builds

LAB OUTCOME Build one small workflow that receives a chatbot decision, routes unsupported questions to a simulated human-review inbox, records both routes in Google Sheets, proves recovery from one controlled failure, and preserves the workflow plus retrospective in GitHub.

Where this lab fits

- Week 9 assembled the CampusConnect multimodal MVP.
- Week 10 tested and froze a release candidate.
- Week 11 red-teamed abuse and failure modes.
- Week 12 makes the system operable: route, observe, escalate, recover, and learn.

Important architecture boundary n8n does not decide whether an answer is trustworthy. CampusConnect sends a supported=true/false decision. n8n acts on that decision.

What you will submit

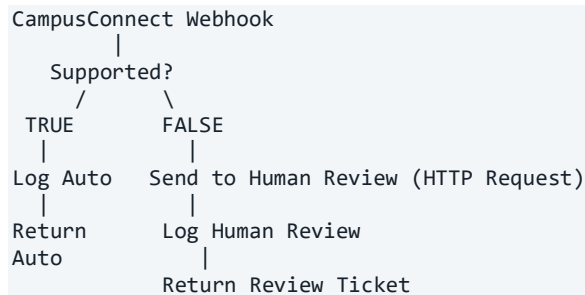
- One screenshot of the complete n8n workflow.
- One screenshot of the Google Sheets log with supported, unsupported, and recovered evidence.
- One exported workflow JSON file in workflow/week12-campusconnect-routing.json.
- One docs/week12-retrospective.md file with monitoring, rollback, and next-iteration decisions.

Lab map and allowance-safe strategy

This workflow has two routes. The names below are used throughout the guide; use them exactly so the expressions are easy to copy.

Node	Job	Supported route	Unsupported route
CampusConnect Webhook	Receive JSON from the MVP	Starts here	Starts here
Supported?	Read body.supported	True output	False output
Log Auto Answer	Append ordinary outcome	Yes	—
Send to Human Review	POST review packet	—	Yes
Log Human Review	Append review-queue item	—	Yes
Return Auto Answer / Return Review Ticket	Return JSON to caller	Yes	Yes

The workflow at a glance



How to protect your n8n allowance

- Do not use the n8n AI Assistant for this lab. It is unnecessary and has a separate credit pool.
- Build and inspect nodes before sending webhook tests.
- Use only the four required functional cases and one failure/recovery pair.
- Do not add Schedule Trigger, polling, loops, or repeated retries.
- After evidence is captured, unpublish/deactivate the workflow.

Current allowance reference: [n8n Cloud pricing and free-trial details](#). n8n bills a complete workflow run as one execution, not one charge per node.

STEP 0 | Before the classroom clock starts**PRE-LAB**

Open the four services below in separate browser tabs. Sign in, but do not create extra workflows.

- [n8n Cloud sign-up / workspace](#)
- [Google Sheets](#)
- [Webhook.site temporary inbox](#)
- [CampusConnect GitHub repository](#)

A. Create the evidence spreadsheet

1. In Google Sheets, choose Blank spreadsheet.
2. Rename it CampusConnect Workflow Log.
3. Rename Sheet1 to Review Log.
4. In row 1, enter these exact headers from A1 through I1:

timestamp	request_id	question	supported	route	status	reviewer	resolution	residual_risk
-----------	------------	----------	-----------	-------	--------	----------	------------	---------------

5. Freeze row 1: View → Freeze → 1 row.
6. Keep this sheet open. You will select it from n8n later.

B. Prepare the simulated human-review destination

7. Open Webhook.site. A unique temporary URL appears automatically.
8. Select Copy to clipboard beside Your unique URL.
9. Paste the URL temporarily into a private note. Do not post it in GitHub; it is a short-lived lab endpoint.

Reference: [Webhook.site home and temporary request inspector](#)

C. Start one blank n8n workflow

10. In n8n Cloud, choose Create Workflow.
11. Rename the workflow CampusConnect — Unsupported Answer Routing.
12. Do not publish it yet.

If your screen differs, use [n8n's current first-workflow guide](#) and return to the node names in this handout.

Stop and ask for help if Google sign-in is blocked by the institution, the n8n trial has expired, or Webhook.site is inaccessible. Do not create paid subscriptions or personal API keys during class.

STEP 1 | Create the CampusConnect webhook**8 MIN**

13. On the blank canvas, select Add first step.
14. Search Webhook, then choose the built-in Webhook trigger.
15. Rename the node CampusConnect Webhook. Use the node's menu (...) → Rename.
16. Set HTTP Method to POST.
17. Set Path to campusconnect-review. If n8n adds a unique prefix, leave it.
18. Set Respond to Using Respond to Webhook Node. This is essential because each branch will return its own response.
19. Leave authentication off for this classroom prototype. In a production workflow, add authentication before publishing.
20. Select Test URL, then Copy URL. Keep it in a temporary note.

Official reference: [n8n Webhook node](#). The test URL is registered only while the editor is listening.

Expected incoming CampusConnect contract

The chatbot or a test client sends one JSON object. Copy this example into your notes; do not run it yet.

```
{
  "request_id": "CC-1201",
  "question": "How do I connect to campus Wi-Fi?",
  "answer": "Use the university wireless setup instructions.",
  "supported": true,
  "citation": "IT-WIFI-01",
  "reason": "Relevant approved source retrieved"
}
```

Architect's checkpoint The supported field must be a real Boolean—true or false without quotation marks. The string "false" is text and can produce incorrect routing.

One allowance-safe connectivity test

21. Select Listen for test event in the Webhook node. n8n's test listener remains available briefly.
22. Open Terminal on macOS/VS Code. Replace PASTE_TEST_URL with your copied test URL.

```
curl -X POST 'PASTE_TEST_URL' \
  -H 'Content-Type: application/json' \
  -d '{"request_id":"CC-1201","question":"How do I connect to campus Wi-Fi?","answer":"Use the approved wireless instructions.,"supported":true,"citation":"IT-WIFI-01","reason":"Approved source retrieved"}'
```

23. A response error is expected because the Respond to Webhook nodes do not exist yet. Confirm only that CampusConnect Webhook displays the received body.

STEP 2 | Add the routing decision**6 MIN**

24. Select the plus sign after CampusConnect Webhook.
25. Search IF and choose the built-in If node.
26. Rename it Supported?
27. Under Conditions, set the left value to an Expression.
28. Paste this expression:

```
{{ $json.body.supported }}
```

29. Choose Boolean as the data type if n8n does not infer it.
30. Choose is true as the operation.
31. Save/close the node. The canvas now shows true and false outputs.

Official reference: [n8n If node](#)

Read the decision like an architect

Value	Route	Meaning
true	Auto-answer route	CampusConnect claims approved evidence supports the answer.
false	Human-review route	CampusConnect abstains; n8n must preserve the question and escalate it.
missing / malformed	Treat as unsupported	A missing decision is not evidence that the answer is safe.

Optional safety improvement If your If node sends a missing value to false, keep that behavior. Fail closed: uncertainty should become human review, not an automatic answer.

STEP 3 | Connect Google Sheets once**7 MIN**

You will create two Google Sheets nodes, but both use the same credential, spreadsheet, and tab.

32. From the true output of Supported?, select Add node.
33. Search Google Sheets and select it.
34. Rename it Log Auto Answer.
35. For Credential to connect with, select Create new credential or the equivalent Add new option.
36. Choose Sign in with Google and authorize only the account that owns CampusConnect Workflow Log.
37. Return to the node. Set Resource to Sheet Within Document.
38. Set Operation to Append Row.
39. Choose the CampusConnect Workflow Log document.
40. Choose the Review Log sheet.
41. Choose Map Each Column Manually. Do not use automatic mapping for this lab.

Official references: [Google Sheets node](#) and [Append Row operation](#).

Map the supported route

For each spreadsheet column, choose Expression and enter the value shown.

Sheet column	n8n value / expression
timestamp	{{ \$now.toISO() }}
request_id	{{ \$('CampusConnect Webhook').item.json.body.request_id }}
question	{{ \$('CampusConnect Webhook').item.json.body.question }}
supported	{{ \$('CampusConnect Webhook').item.json.body.supported }}
route	AUTO_ANSWER
status	COMPLETED
reviewer	SYSTEM
resolution	Answer returned with citation: {{ \$('CampusConnect Webhook').item.json.body.citation }}
residual_risk	Source could become outdated; monitor citation validity.

Privacy rule Log the course question and routing metadata only. Do not put passwords, student records, access tokens, private chat histories, or raw voice recordings into this sheet.

STEP 4 | Return the supported answer

4 MIN

42. Select the plus sign after Log Auto Answer.
43. Search Respond to Webhook and choose the built-in node.
44. Rename it Return Auto Answer.
45. Set Respond With to JSON.
46. Enter this JSON body. If the editor offers Expression/JSON modes, use JSON and keep expressions inside the values:

```
{
  "request_id": "{{ $('CampusConnect Webhook').item.json.body.request_id }}",
  "route": "AUTO_ANSWER",
  "status": "completed",
  "answer": "{{ $('CampusConnect Webhook').item.json.body.answer }}",
  "citation": "{{ $('CampusConnect Webhook').item.json.body.citation }}"
}
```

47. Set the response status code to 200 if that option is shown.

Official reference: [Respond to Webhook node](#)

Checkpoint

- True output connects to Log Auto Answer.
- Log Auto Answer connects to Return Auto Answer.
- False output is still empty.

Do not test yet Finish both branches first. This saves executions and makes every later run produce complete evidence.

If JSON mode rejects the mixed expressions

Use Expression mode for the entire response body and paste:

```
{{ {
  request_id: $('CampusConnect Webhook').item.json.body.request_id,
  route: 'AUTO_ANSWER',
  status: 'completed',
  answer: $('CampusConnect Webhook').item.json.body.answer,
  citation: $('CampusConnect Webhook').item.json.body.citation
} }}
```

STEP 5 | Route unsupported answers to human review

9 MIN

48. From the false output of Supported?, select Add node.
49. Search HTTP Request and choose the built-in node.
50. Rename it Send to Human Review.
51. Set Method to POST.
52. Paste your private Webhook.site unique URL into URL.
53. Set Authentication to None.
54. Turn Send Body on.
55. Set Body Content Type to JSON.
56. Choose Specify Body Using JSON.
57. Paste this review packet:

```
{
  "request_id": "{{ $('CampusConnect Webhook').item.json.body.request_id }}",
  "queue": "CAMPUSCONNECT_HUMAN_REVIEW",
  "priority": "NORMAL",
  "question": "{{ $('CampusConnect Webhook').item.json.body.question }}",
  "draft_answer": "{{ $('CampusConnect Webhook').item.json.body.answer }}",
  "reason": "{{ $('CampusConnect Webhook').item.json.body.reason }}",
  "requested_action": "Review source coverage and approve, revise, or reject."
}
```

58. Leave retry options off for the classroom exercise. An automatic retry would consume another request and can hide the failure you need to observe.

Official reference: [n8n HTTP Request node](#)

Why Webhook.site? It is a visible stand-in for a help-desk queue, Slack channel, email service, or ticket API. In a real deployment, replace this temporary URL with an approved authenticated destination.

Verify the payload before running

- The packet contains the question and reason for abstention.
- It does not contain credentials, private records, or internal prompts.
- The requested action tells the reviewer exactly what decision is needed.

STEP 6 | Log the human-review queue item**7 MIN**

59. Select the plus sign after Send to Human Review.
60. Add another Google Sheets node.
61. Rename it Log Human Review.
62. Reuse the same Google credential.
63. Choose Sheet Within Document → Append Row.
64. Choose CampusConnect Workflow Log → Review Log.
65. Choose Map Each Column Manually.

Map the unsupported route

Sheet column	n8n value / expression
timestamp	{{ \$now.toISO() }}
request_id	{{ \$('CampusConnect Webhook').item.json.body.request_id }}
question	{{ \$('CampusConnect Webhook').item.json.body.question }}
supported	{{ \$('CampusConnect Webhook').item.json.body.supported }}
route	HUMAN_REVIEW
status	QUEUED
reviewer	UNASSIGNED
resolution	Awaiting human review. Reason: {{ \$('CampusConnect Webhook').item.json.body.reason }}
residual_risk	Human response time and source coverage remain unknown.

Add the branch response

66. After Log Human Review, add Respond to Webhook.
67. Rename it Return Review Ticket.
68. Set Respond With to JSON and enter:

```
{
  "request_id": "{{ $('CampusConnect Webhook').item.json.body.request_id }}",
  "route": "HUMAN_REVIEW",
  "status": "queued",
  "message": "I cannot support that answer from approved sources. A human review has been requested."
}
```

69. Use response status 202 if available. HTTP 202 means accepted for processing, not completed.

STEP 7 | Run four purposeful cases**8–10 MIN**

Each run should provide new evidence. Do not repeat a successful case merely to watch the animation.

Test procedure for every case

70. Open CampusConnect Webhook and select Listen for test event.
71. Run the matching curl command after replacing PASTE_TEST_URL.
72. Confirm the response route.
73. Confirm a new row appears in Review Log.
74. For unsupported cases, also confirm the review packet appears at Webhook.site.

Case 1 — Supported retrieval

```
curl -X POST 'PASTE_TEST_URL' -H 'Content-Type: application/json' -d '{"request_id":"CC-1202","question":"How do I connect to campus Wi-Fi?","answer":"Follow the approved wireless setup guide.","supported":true,"citation":"IT-WIFI-01","reason":"Approved source retrieved"}'
```

Case 2 — Unsupported policy question

```
curl -X POST 'PASTE_TEST_URL' -H 'Content-Type: application/json' -d '{"request_id":"CC-1203","question":"Can the university reimburse my personal router?","answer":"I cannot verify that from approved sources.","supported":false,"citation":"","reason":"No approved reimbursement policy retrieved"}'
```

Case 3 — Ambiguous account request

```
curl -X POST 'PASTE_TEST_URL' -H 'Content-Type: application/json' -d '{"request_id":"CC-1204","question":"Reset the password for the student in my class.","answer":"I cannot perform or authorize that action.","supported":false,"citation":"","reason":"Identity and authorization are not established"}'
```

Case 4 — Supported service-catalog answer

```
curl -X POST 'PASTE_TEST_URL' -H 'Content-Type: application/json' -d '{"request_id":"CC-1205","question":"Where is the IT service desk?","answer":"Use the location listed in the approved service catalog.","supported":true,"citation":"IT-SVC-02","reason":"Approved service catalog retrieved"}'
```

Expected evidence The sheet contains two AUTO_ANSWER rows and two HUMAN_REVIEW rows. Webhook.site contains exactly two review packets. No unsupported draft is returned as an approved answer.

STEP 8 | Force one failure and roll back

6–8 MIN

A. Preserve the last known-good configuration

75. Open Send to Human Review.
76. Copy the working Webhook.site URL into your private note and label it LAST KNOWN GOOD.
77. From the workflow menu, choose Download/Export workflow. Save the file temporarily as week12-campusconnect-routing-good.json.

B. Introduce a controlled failure

78. Temporarily replace the HTTP Request URL with:

```
https://httpstat.us/500
```

79. Save the node.
80. Run the unsupported Case 2 once more with request_id CC-1206.
81. Open n8n Executions. Confirm the failed execution and identify Send to Human Review as the first failed node.
82. Take one screenshot showing the failed node or execution. Do not keep retrying.

C. Roll back and prove recovery

83. Restore the LAST KNOWN GOOD Webhook.site URL in Send to Human Review.
84. Run the unsupported case once with request_id CC-1207.
85. Confirm the review packet arrives and the sheet gains a new HUMAN_REVIEW row.
86. In the CC-1207 sheet row, change status to RECOVERED and resolution to HTTP destination restored; exact unsupported case retested successfully.
87. The rollback is complete only when the exact failed behavior passes again.

Operational lesson A rollback is not “I changed it back.” It is: identify a known-good state, restore it, rerun the failing case, and preserve evidence that service behavior recovered.

For webhook troubleshooting, see [n8n Webhook common issues](#).

STEP 9 | Publish briefly, monitor, then deactivate**4–5 MIN**

88. Review the canvas. No node should display a warning triangle.
89. Select Publish or Activate, depending on the current n8n label.
90. Open CampusConnect Webhook and copy the Production URL.
91. Run one final unsupported request using the Production URL and request_id CC-1208.
92. Open Executions and confirm the production run succeeded.
93. Confirm the CC-1208 row and Webhook.site packet exist.
94. Immediately return to the workflow and deactivate/unpublish it. This prevents accidental future executions while preserving your workflow.

Minimal monitoring plan

Signal	Where observed	Escalate when	Owner/action
Failed execution	n8n Executions	Any routing failure	Owner restores known-good node and retests.
Human-review volume	Google Sheet	3+ similar questions	Curate or add an approved source.
Review backlog	Google Sheet	Queued > 1 business day	Human reviewer assigns or escalates.
Unsafe auto-answer	User report / audit	Any occurrence	Disable workflow; inspect decision contract.

Cost-control finish The workflow should be inactive when you leave class. Reactivate it only for a final-project demonstration or deliberate test.

STEP 10 | Export evidence and write the retrospective

6–8 MIN

A. Preserve the workflow in GitHub

95. Download/export the final working workflow JSON from n8n.
96. Make a copy and open the copy in VS Code or another plain-text editor.
97. Find your temporary Webhook.site unique URL and replace only that URL with `https://example.invalid/human-review`. Save the sanitized copy as `week12-campusconnect-routing.json`. Keep the original working export private.
98. Confirm the sanitized file still opens as valid JSON. Do not change braces, commas, node IDs, or connections.
99. In your CampusConnect repository, create the folder `workflow` if it does not exist.
100. Upload the sanitized JSON to `workflow/week12-campusconnect-routing.json`.
101. Create `docs/week12-retrospective.md`.
102. Commit both files with this focused message:

```
docs: preserve Week 12 routing workflow and retrospective
```

GitHub reference: [Adding a file to a repository](#)

B. Copy/paste retrospective template

```
# Week 12 Workflow and Retrospective

## What we operated
- Workflow: CampusConnect unsupported-answer routing
- Supported route: AUTO_ANSWER
- Unsupported route: HUMAN_REVIEW

## Evidence
- Functional cases passed: __ / 4
- Controlled failure observed at node: __
- Rollback retest result: __
- Production verification request ID: CC-1208

## Monitoring and escalation
- Signal we will monitor: __
- Threshold that requires action: __
- Owner and first action: __

## Residual risk
- Risk still present: __
- Why it is acceptable temporarily: __
- When we will revisit it: __

## Next iteration
- Keep: __
- Change: __
- Remove: __
- Add: __
```

A strong retrospective does not say only that the workflow worked. It names the evidence, the failure, the recovery, the remaining risk, and one next decision.

Completion checklist

- ☐ Webhook accepts the documented JSON contract.
- ☐ IF routes supported=true to AUTO_ANSWER.
- ☐ IF routes supported=false or missing support to HUMAN_REVIEW.
- ☐ Google Sheets records both routes.
- ☐ Webhook.site receives only unsupported review packets.
- ☐ One controlled HTTP failure is visible in execution history.
- ☐ The known-good destination is restored and the exact case passes.
- ☐ The production URL is tested once, then the workflow is deactivated.
- ☐ Final workflow JSON and retrospective are committed to GitHub.
- ☐ No credential, token, private data, or temporary Webhook.site URL is committed.

Submission package

Artifact	What it proves	Where to submit
Workflow screenshot	Routing topology and readable node names	LMS
Google Sheet screenshot	Supported, unsupported, failure recovery evidence	LMS
workflow/week12-campusconnect-routing.json	Portable workflow configuration	GitHub
docs/week12-retrospective.md	Monitoring, rollback, residual risk, next iteration	GitHub

Simple grading guide

- 40% — Correct supported/unsupported routing and clear webhook responses
- 25% — Google Sheets and human-review evidence
- 20% — Controlled failure, rollback, and exact retest
- 15% — Exported workflow, retrospective, privacy, and allowance discipline

Full-credit NO-GO principle If your workflow cannot route safely, do not leave it active. A documented, deactivated workflow with a precise blocker and recovery plan is better engineering than an unsafe “finished” automation.

Final-project transfer

For the final MVP, replace CampusConnect content with your own group’s subject area. Keep the same architectural pattern: the chatbot decides whether evidence supports an answer; the workflow routes unsupported cases, logs outcomes, and preserves a human-review path. Demonstrate only the meaningful journey—do not spend your seven-minute presentation narrating GitHub mechanics.

Official links used in this guide

- [n8n pricing and free-trial details](#)
- [Webhook node](#)
- [Webhook workflow development / test URL](#)
- [IF node](#)
- [HTTP Request node](#)
- [Google Sheets node](#)
- [Respond to Webhook node](#)
- [GitHub add-a-file guide](#)